

K L Deemed to be University

Department of Computer Science and Engineering

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 8 – The Transactional Microservice

Name	Chakrika Panakala
Roll No.	2520090042
Course	25CS1302E
CO Mapping	CO4
Session	Practical 8

Aim

To build a Spring Boot inventory-service (Spring Web, Spring Data JPA, PostgreSQL Driver, Validation) with Inventory and Order entities, JpaRepository repositories, a service that allows a borrow only when availableStock > 0, POST /orders and GET /inventory/{bookId} endpoints, JSR-380 validation with a @ControllerAdvice error handler, and to route requests to it through the FastAPI Gateway.

Program

The project is implemented as a Spring Boot microservice using Java 17 and Maven. PostgreSQL stores the inventory and order information. A FastAPI gateway is used to expose the service through a common API entry point.

1. Maven Configuration (pom.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>
  <groupId>com.bookflow</groupId>
  <artifactId>inventory-service</artifactId>
  <version>1.0.0</version>
  <name>inventory-service</name>
  <description>BookFlow inventory and order microservice</description>
  <properties><java.version>17</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId></dependency>
    <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId><scope>test</scope></dependency>
  </dependencies>
  <build><plugins><plugin>
    <groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId>
  </plugin></plugins></build>
</project>
```

2. Database Configuration (application.yml)

```
server:
  port: 8080
spring:
  application:
    name: inventory-service
  datasource:
    url: jdbc:postgresql://localhost:5432/bookflow_db
```

```

    username: postgres
    password: postgres
    driver-class-name: org.postgresql.Driver
jpa:
  hibernate:
    ddl-auto: update
    show-sql: false
    open-in-view: false
  properties:
    hibernate:
      format_sql: true

```

3. Main Spring Boot Class

```

package com.bookflow.inventory;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class InventoryServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(InventoryServiceApplication.class, args);
    }
}

```

4. Inventory Entity

```

package com.bookflow.inventory.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;

@Entity
@Table(name = "inventory")
public class Inventory {
    @Id
    @Column(name = "book_id")
    private Long bookId;

    @NotNull
    @Min(value = 0, message = "totalStock cannot be negative")
    @Column(name = "total_stock", nullable = false)
    private Integer totalStock;

    @NotNull
    @Min(value = 0, message = "availableStock cannot be negative")
    @Column(name = "available_stock", nullable = false)
    private Integer availableStock;

    protected Inventory() {}

    public Inventory(Long bookId, Integer totalStock, Integer availableStock) {
        this.bookId = bookId;
        this.totalStock = totalStock;
        this.availableStock = availableStock;
    }

    public Long getBookId() { return bookId; }
    public Integer getTotalStock() { return totalStock; }
    public Integer getAvailableStock() { return availableStock; }
    public void setTotalStock(Integer totalStock) { this.totalStock = totalStock; }
    public void setAvailableStock(Integer availableStock) { this.availableStock = availableStock; }
}

```

5. Order Entity and Status

```

package com.bookflow.inventory.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "orders")
public class BookOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id", nullable = false, length = 24)
    private String userId;

    @Column(name = "book_id", nullable = false)
    private Long bookId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)

```

```

private OrderStatus status;

@Column(name = "borrowed_at", nullable = false)
private LocalDateTime borrowedAt;

@Column(name = "returned_at")
private LocalDateTime returnedAt;

protected BookOrder() {}

public BookOrder(String userId, Long bookId) {
    this.userId = userId;
    this.bookId = bookId;
    this.status = OrderStatus.BORROWED;
    this.borrowedAt = LocalDateTime.now();
}

public void markReturned() {
    this.status = OrderStatus.RETURNED;
    this.returnedAt = LocalDateTime.now();
}

public Long getId() { return id; }
public String getUserId() { return userId; }
public Long getBookId() { return bookId; }
public OrderStatus getStatus() { return status; }
public LocalDateTime getBorrowedAt() { return borrowedAt; }
public LocalDateTime getReturnedAt() { return returnedAt; }
}

public enum OrderStatus {
    BORROWED, RETURNED
}

```

6. Repository Layer

```

package com.bookflow.inventory.repository;

import com.bookflow.inventory.entity.Inventory;
import jakarta.persistence.LockModeType;
import java.util.Optional;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;

public interface InventoryRepository extends JpaRepository<Inventory, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT i FROM Inventory i WHERE i.bookId = :bookId")
    Optional<Inventory> findByIdForUpdate(@Param("bookId") Long bookId);
}

public interface OrderRepository extends JpaRepository<BookOrder, Long> {
    List<BookOrder> findByUserIdOrderByBorrowedAtDesc(String userId);
}

```

7. Request DTOs

```

package com.bookflow.inventory.dto;

import jakarta.validation.constraints.*;

public record OrderRequest(
    @NotBlank(message = "userId is required") String userId,
    @NotNull(message = "bookId is required")
    @Positive(message = "bookId must be positive") Long bookId) {}

public record InventoryRequest(
    @NotNull(message = "bookId is required")
    @Positive(message = "bookId must be positive") Long bookId,
    @NotNull(message = "totalStock is required")
    @Min(value = 0, message = "totalStock cannot be negative") Integer totalStock) {}

```

8. Transactional Service Layer

```

@Service
public class LibraryService {
    private final InventoryRepository inventoryRepository;
    private final OrderRepository orderRepository;

    public LibraryService(InventoryRepository inventoryRepository,
        OrderRepository orderRepository) {
        this.inventoryRepository = inventoryRepository;
        this.orderRepository = orderRepository;
    }

    @Transactional(readOnly = true)
    public Inventory getInventory(Long bookId) {
        return inventoryRepository.findById(bookId)
            .orElseThrow(() ->

```

```

        new ResourceNotFoundException("No inventory record for book " + bookId));
    }

    @Transactional
    public BookOrder borrowBook(OrderRequest request) {
        Inventory inventory = inventoryRepository.findByIdForUpdate(request.bookId())
            .orElseThrow(() ->
                new ResourceNotFoundException("No inventory record for book " + request.bookId()));

        if (inventory.getAvailableStock() <= 0) {
            throw new OutOfStockException(request.bookId());
        }

        inventory.setAvailableStock(inventory.getAvailableStock() - 1);
        return orderRepository.save(
            new BookOrder(request.userId(), request.bookId()));
    }

    @Transactional
    public BookOrder returnBook(Long orderId) {
        BookOrder order = orderRepository.findById(orderId)
            .orElseThrow(() ->
                new ResourceNotFoundException("Order " + orderId + " not found"));

        if (order.getStatus() == OrderStatus.RETURNED) {
            throw new InvalidOperationException(
                "Order " + orderId + " has already been returned");
        }

        Inventory inventory = inventoryRepository.findByIdForUpdate(order.getBookId())
            .orElseThrow(() ->
                new ResourceNotFoundException("No inventory record for book " + order.getBookId()));

        inventory.setAvailableStock(inventory.getAvailableStock() + 1);
        order.markReturned();
        return order;
    }
}

```

9. REST Controllers

```

@RestController
@RequestMapping("/inventory")
public class InventoryController {
    @GetMapping("/{bookId}")
    public Inventory getInventory(@PathVariable Long bookId) {
        return libraryService.getInventory(bookId);
    }

    @PostMapping
    public ResponseEntity<Inventory> addStock(
        @Valid @RequestBody InventoryRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(libraryService.addOrUpdateStock(request));
    }
}

@RestController
@RequestMapping("/orders")
public class OrderController {
    @PostMapping
    public ResponseEntity<BookOrder> borrow(
        @Valid @RequestBody OrderRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(libraryService.borrowBook(request));
    }

    @PutMapping("/{orderId}/return")
    public BookOrder returnBook(@PathVariable Long orderId) {
        return libraryService.returnBook(orderId);
    }

    @GetMapping
    public List<BookOrder> ordersForUser(@RequestParam String userId) {
        return libraryService.ordersForUser(userId);
    }
}

```

10. Exception Handling

```

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OutOfStockException.class)
    public ResponseEntity<Map<String, Object>> handleOutOfStock(
        OutOfStockException ex) {
        return build(HttpStatus.CONFLICT, ex.getMessage(), null);
    }

    @ExceptionHandler(ResourceNotFoundException.class)

```

```

public ResponseEntity<Map<String, Object>> handleNotFound(
    ResourceNotFoundException ex) {
    return build(HttpStatus.NOT_FOUND, ex.getMessage(), null);
}

@ExceptionHandler(InvalidOperationException.class)
public ResponseEntity<Map<String, Object>> handleInvalid(
    InvalidOperationException ex) {
    return build(HttpStatus.BAD_REQUEST, ex.getMessage(), null);
}

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidation(
    MethodArgumentNotValidException ex) {
    List<String> details = ex.getBindingResult().getFieldErrors().stream()
        .map(error -> error.getField() + ": " + error.getDefaultMessage())
        .sorted().toList();
    return build(HttpStatus.BAD_REQUEST, "Validation failed", details);
}
}

```

11. FastAPI Gateway

```

import os
from typing import Any, Optional
import httpx
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse, Response

INVENTORY_SERVICE_URL = os.getenv(
    "INVENTORY_SERVICE_URL", "http://localhost:8080")
TIMEOUT = httpx.Timeout(5.0, connect=2.0)

app = FastAPI(title="BookFlow Gateway", version="1.1.0")

async def forward(method: str, url: str,
                  json_body: Optional[Any] = None) -> Response:
    try:
        async with httpx.AsyncClient(timeout=TIMEOUT) as client:
            upstream = await client.request(method, url, json=json_body)
    except (httpx.ConnectError, httpx.TimeoutException):
        return JSONResponse(
            status_code=503,
            content={"error": "Service Unavailable",
                    "message": "Inventory service is not reachable. Please try again later."}
        )

    return Response(
        content=upstream.content,
        status_code=upstream.status_code,
        media_type=upstream.headers.get("content-type", "application/json"),
    )

@app.get("/api/v1/inventory/{book_id}")
async def get_inventory(book_id: int):
    return await forward("GET", f"{INVENTORY_SERVICE_URL}/inventory/{book_id}")

@app.post("/api/v1/inventory")
async def add_stock(request: Request):
    return await forward("POST", f"{INVENTORY_SERVICE_URL}/inventory",
                        await request.json())

@app.post("/api/v1/orders")
async def borrow_book(request: Request):
    return await forward("POST", f"{INVENTORY_SERVICE_URL}/orders",
                        await request.json())

@app.put("/api/v1/orders/{order_id}/return")
async def return_book(order_id: int):
    return await forward(
        "PUT", f"{INVENTORY_SERVICE_URL}/orders/{order_id}/return")

```

Output

The following results are based on the supplied practical reference. Runtime-generated timestamps and database IDs may differ when the program is executed again.

1. Add inventory

Endpoint: POST /api/v1/inventory

```
{"bookId":3,"totalStock":2}
```

Observed result: HTTP 201 — availableStock becomes 2.

2. Check inventory

Endpoint: GET /api/v1/inventory/3

Observed result: HTTP 200 — {"bookId":3,"totalStock":2,"availableStock":2}

3. First borrow

Endpoint: POST /api/v1/orders

```
{"userId":"6abf9f1c629a41fd16542538","bookId":3}
```

Observed result: HTTP 201 — order status is BORROWED.

4. Second borrow

Endpoint: POST /api/v1/orders

```
{"userId":"6abf9f1c629a41fd16542537","bookId":3}
```

Observed result: HTTP 201 — second copy is borrowed.

5. Stock after two borrows

Endpoint: GET /api/v1/inventory/3

Observed result: HTTP 200 — availableStock is 0.

6. Third borrow

Endpoint: POST /api/v1/orders

```
{"userId":"6abf9f1c629a41fd16542539","bookId":3}
```

Observed result: HTTP 409 — Book 3 is out of stock.

7. Validation

Endpoint: POST /api/v1/orders

```
{"userId":"","bookId":-1}
```

Observed result: HTTP 400 — validation failed for userId and bookId.

8. Missing inventory

Endpoint: GET /api/v1/inventory/99

Observed result: HTTP 404 — no inventory record for book 99.

9. Return order

Endpoint: PUT /api/v1/orders/1/return

Observed result: HTTP 200 — order becomes RETURNED and availableStock becomes 1.

10. Gateway when service is down

Endpoint: GET /api/v1/inventory/3

Observed result: HTTP 503 — Inventory service is not reachable.

Database Verification

```
bookflow_db=# SELECT * FROM inventory;
book_id | available_stock | total_stock
3       | 1               | 2
```

```
bookflow_db=# SELECT id, user_id, book_id, status FROM orders ORDER BY id;
id | user_id | book_id | status
1  | 6abf9f1c629a41fd16542538 | 3       | RETURNED
2  | 6abf9f1c629a41fd16542537 | 3       | BORROWED
```

Result

The Spring Boot inventory service successfully handles stock and borrowing operations through JPA-backed transactions. The borrow operation decreases available stock only when a copy exists, while the row lock protects the last available copy from conflicting updates. Validation and centralized exception handling return structured HTTP errors, and the FastAPI Gateway forwards requests to the Spring Boot service and reports a 503 response when the downstream service is unavailable. The PostgreSQL tables are created and updated through Spring Data JPA.

Reference: Practical Experiment 8 supplied for this assignment.